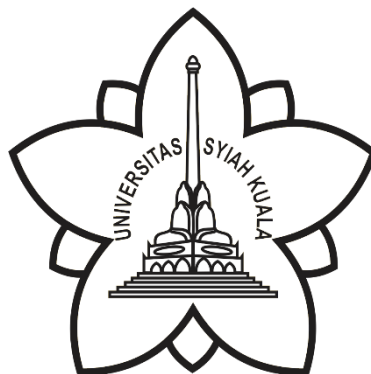


TUGAS 3 LOGISTIC REGRESSION, K-NEAREST NEIGHBOR, NAIVE BAYES, DECISION TREE

Disusun untuk memenuhi
Tugas Mata Kuliah Pembelajaran Mesin

Oleh :

Iffatun Nisa Nasrullah	(2208107010009)
Muhammad Bintang Indra Hidayat	(2208107010023)
Qandila Ahmara	(2208107010039)
Alhusna Hanifah	(2208107010060)
Farhanul Khair	(2208107010076)



**JURUSAN INFORMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS SYIAH KUALA
DARUSSALAM, BANDA ACEH**

2025

1. Pemahaman Dataset

Dataset: Metabolic Syndrome

Sumber:

Kaggle

<https://www.kaggle.com/datasets/antimoni/metabolic-syndrome/data>

Tahun rilis dataset: 2023

Pernah digunakan oleh pihak lain:

https://www.researchgate.net/publication/379039640_Perbandingan_Algoritma_Naive_Bayes_dan_K-Nearest_Neighbors_untuk_Klasifikasi_Metabolik_Sindrom_Comparison_of_Naive_Bayes_and_K-Nearest_Neighbors_Algorithms_for_Metabolic_Syndrome_Classification

Variabel yang digunakan:

- **seqn:** Nomor identifikasiurut individu.
- **Age:** Usia individu.
- **Sex:** Jenis kelamin individu (misalnya, Laki-laki, Perempuan).
- **Marital:** Status pernikahan individu.
- **Income:** Tingkat pendapatan atau informasi terkait pendapatan.
- **Race:** Latar belakang etnis atau ras individu.
- **WaistCirc:** Pengukuran lingkarpinggang individu.
- **BMI:** Indeks Massa Tubuh, sebuah ukuran komposisi tubuh yang dihitung berdasarkan berat badan dan tinggi badan.
- **Albuminuria:** Pengukuran terkait albumin dalam urin, yang bisa menjadi indikator adanya kerusakan ginjal.
- **UrAlbCr:** Rasio albumin terhadap kreatinin urin, digunakan untuk menilai fungsi ginjal.
- **UricAcid:** Kadar asam urat dalam darah, yang dapat berhubungan dengan risiko gout atau masalah metabolik lainnya.
- **BloodGlucose:** Kadar glukosa dalam darah, yang merupakan indikator penting untuk risiko diabetes.
- **HDL:** Kadar kolesterol High-Density Lipoprotein (HDL), yang sering disebut sebagai "kolesterol baik" karena perannya dalam mengurangi risiko penyakit jantung.
- **Triglycerides:** Kadar trigliserida dalam darah, yang dapat berhubungan dengan risiko penyakit kardiovaskular.
- **MetabolicSyndrome:** Variabel biner yang menunjukkan keberadaan (1) atau ketidakhadiran (0) sindrom metabolik, suatu kondisi yang melibatkan beberapa faktor risiko penyakit jantung dan diabetes.

Gambar di bawah ini menunjukkan proses awal dalam pengolahan dataset, yaitu memuat data dari sumber yang telah ditentukan ke dalam lingkungan kerja Python menggunakan pustaka seperti pandas. Data yang digunakan adalah dataset "Metabolic Syndrome" yang kemudian akan digunakan untuk proses eksplorasi dan pemodelan.

```
df=pd.read_csv('/content/metabolic-syndrome-data/Metabolic Syndrome.csv')
df.head()
```

	seqn	Age	Sex	Marital	Income	Race	WaistCirc	BMI	Albuminuria	UrAlbCr	UricAcid	BloodGlucose	HDL	Triglycerides	MetabolicSyndrome
0	62161	22	Male	Single	8200.0	White	81.0	23.3	0	3.88	4.9	92	41	84	0
1	62164	44	Female	Married	4500.0	White	80.1	23.2	0	8.55	4.5	82	28	56	0
2	62169	21	Male	Single	800.0	Asian	69.6	20.1	0	5.07	5.4	107	43	78	0
3	62172	43	Female	Single	2000.0	Black	120.4	33.3	0	5.22	5.0	104	73	141	0
4	62177	51	Male	Married	NaN	Asian	81.1	20.1	0	8.13	5.0	95	43	126	0

Gambar 1.1 Loading Data

Statistik Deskriptif dan Visualisasi

```
df.info()
```

<class 'pandas.core.frame.DataFrame'>			
RangeIndex: 2401 entries, 0 to 2400			
Data columns (total 15 columns):			
#	Column	Non-Null Count	Dtype
0	seqn	2401 non-null	int64
1	Age	2401 non-null	int64
2	Sex	2401 non-null	object
3	Marital	2193 non-null	object
4	Income	2284 non-null	float64
5	Race	2401 non-null	object
6	WaistCirc	2316 non-null	float64
7	BMI	2375 non-null	float64
8	Albuminuria	2401 non-null	int64
9	UrAlbCr	2401 non-null	float64
10	UricAcid	2401 non-null	float64
11	BloodGlucose	2401 non-null	int64
12	HDL	2401 non-null	int64
13	Triglycerides	2401 non-null	int64
14	MetabolicSyndrome	2401 non-null	int64
dtypes: float64(5), int64(7), object(3)			
memory usage: 281.5+ KB			

Gambar 1.2 Statistik Data

Gambar ini menampilkan ringkasan statistik deskriptif dari dataset, termasuk nilai rata-rata (mean), standar deviasi (std), nilai minimum dan maksimum, serta kuartil untuk setiap kolom numerik. Statistik ini membantu dalam memahami distribusi dan skala data sebelum dilakukan analisis lebih lanjut.

```

columns_to_look = ["Sex", "Marital", "Race", "Albuminuria", "MetabolicSyndrome"]
for x in columns_to_look:
    fig, ax = plt.subplots(1, 2, figsize=(16, 5))
    ax = ax.flatten()

    value_counts = df[x].value_counts()
    labels = value_counts.index.tolist()

    pie_colors = sns.color_palette('viridis', len(labels))
    ax[0].pie(
        value_counts,
        shadow=True,
        explode=[.1 for _ in range(value_counts.nunique())],
        autopct='%1.1f%%',
        textprops={'size': 14, 'color': 'white'},
        colors=pie_colors
    )
    ax[0].legend(labels, loc=(1.0, 0.5))

    #####

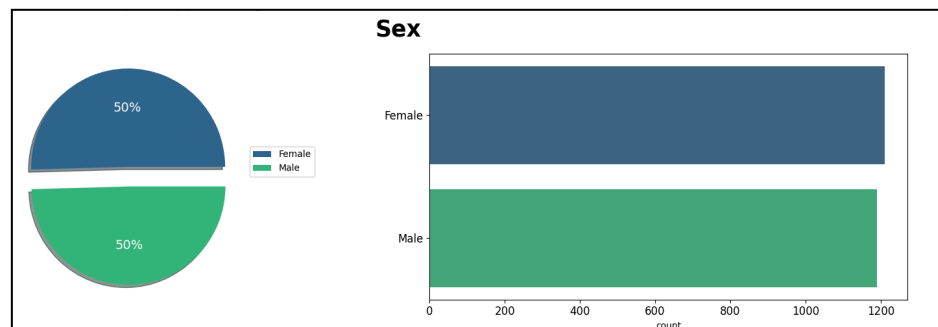
    sns.countplot(data=df, y=x, ax=ax[1], palette=pie_colors, order=labels)
    ax[1].yaxis.label.set_size(20)
    plt.yticks(fontsize=12)
    #ax[1].set_xlabel('Count', fontsize=20)
    ax[1].set_ylabel(None)
    plt.xticks(fontsize=12)

    fig.suptitle(x, fontsize=25, fontweight='bold')
    plt.tight_layout()
    plt.show()

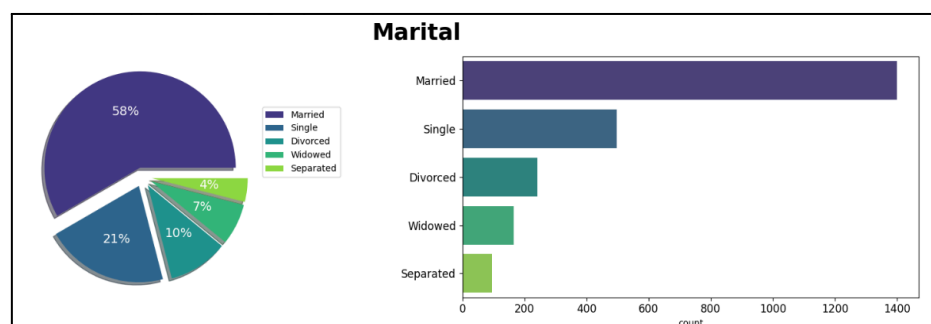
```

Gambar 1.3 Kode untuk menampilkan statistik deskriptif.

Gambar ini menampilkan cuplikan kode Python yang digunakan untuk menghasilkan ringkasan statistik pada Gambar 1.2, biasanya dengan fungsi `describe()` dari pustaka `pandas`.



Gambar 1.4 Distribusi Responden Berdasarkan Jenis Kelamin



Gambar 1.4 Distribusi Responden Berdasarkan Status Pernikahan

Visualisasi ini menggambarkan distribusi jenis kelamin dalam dataset, antara laki-laki dan perempuan. Tujuannya adalah untuk melihat proporsi data berdasarkan jenis kelamin dan apakah terdapat ketidakseimbangan yang signifikan.

2. Eksplorasi Data dan Pra-pemrosesan

```
# Calculate descriptive statistics for the numerical columns
df.describe()
```

	seqn	Age	Income	WaistCirc	BMI	Albuminuria	UrAlbCr	UricAcid	BloodGlucose	HDL	Triglycerides	MetabolicSyndrome
count	2401.000000	2401.000000	2284.000000	2316.000000	2375.000000	2401.000000	2401.000000	2401.000000	2401.000000	2401.000000	2401.000000	2401.000000
mean	67030.674302	48.691795	4005.253940	98.307254	28.702189	0.154102	43.626131	5.489046	108.247813	53.369429	128.125364	0.342357
std	2823.565114	17.632852	2954.032186	16.252634	6.662242	0.422780	258.272829	1.439358	34.820657	15.185537	95.322477	0.474597
min	62161.000000	20.000000	300.000000	56.200000	13.400000	0.000000	1.400000	1.800000	39.000000	14.000000	26.000000	0.000000
25%	64591.000000	34.000000	1600.000000	86.675000	24.000000	0.000000	4.450000	4.500000	92.000000	43.000000	75.000000	0.000000
50%	67059.000000	48.000000	2500.000000	97.000000	27.700000	0.000000	7.070000	5.400000	99.000000	51.000000	103.000000	0.000000
75%	69495.000000	63.000000	6200.000000	107.625000	32.100000	0.000000	13.690000	6.400000	110.000000	62.000000	150.000000	1.000000
max	71915.000000	80.000000	9000.000000	176.000000	68.700000	2.000000	5928.000000	11.300000	382.000000	156.000000	1562.000000	1.000000

```
# Check for missing values in the dataset
missing_values = df.isnull().sum()

missing_values
```

	0
seqn	0
Age	0
Sex	0
Marital	208
Income	117
Race	0
WaistCirc	85
BMI	26
Albuminuria	0
UrAlbCr	0
UricAcid	0
BloodGlucose	0
HDL	0
Triglycerides	0
MetabolicSyndrome	0

```
dtype: int64
```

Pra-Pemrosesan

```
# Pipeline untuk kolom numerik: imputasi + scaling
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy="mean")),
    ('scaler', RobustScaler())
])
```

Gambar 2.3 Pipeline untuk kolom numerik

Gambar ini menunjukkan bagaimana pipeline prapemrosesan dibuat untuk kolom numerik, termasuk proses imputasi nilai yang hilang dan normalisasi atau standardisasi fitur agar skala data seragam sebelum masuk ke model pembelajaran mesin.

```
# Pipeline untuk kolom kategorikal: imputasi + one-hot encoding
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy="most_frequent")),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])
```

Gambar 2.4 Pipeline untuk kolom kategorikal

Gambar ini memperlihatkan pipeline untuk menangani data kategorikal. Biasanya mencakup tahap imputasi dengan modus dan transformasi seperti encoding menggunakan One-Hot Encoding atau Ordinal Encoding untuk mengubah kategori menjadi format numerik.

```
# Label encoding for the target variable
le = LabelEncoder()
y_train_preprocessed = le.fit_transform(y_train)
y_test_preprocessed = le.transform(y_test)

# Apply preprocessing
X_train_preprocessed = preprocessor.fit_transform(X_train)
X_test_preprocessed = preprocessor.transform(X_test)
```

Gambar 2.5 Label encoding

Gambar ini menunjukkan proses label encoding, yakni mengonversi label kategorikal ke bentuk angka. Proses ini penting agar model pembelajaran mesin dapat memahami dan memproses variabel target atau prediktor yang berupa kategori.

EDA

```
def plot_numeric_kde_boxplot(train_df, numeric_cols):
    for col in numeric_cols:
        fig, axes = plt.subplots(1, 2, figsize=(18, 5))

        # Remove palette=default_palette and use default coloring
        sns.histplot(train_df, x=col, kde=True,
                     color='orange', ax=axes[0], label=f'{col}')

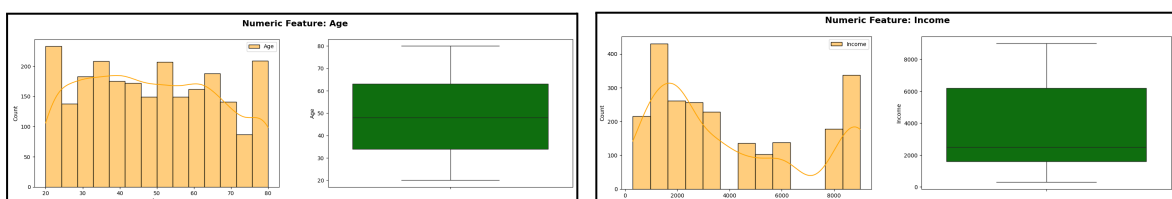
        sns.boxplot(train_df, y=col, color='green', ax=axes[1])
        axes[0].legend()

        fig.suptitle(f'Numeric Feature: {col}', fontsize=16, fontweight='bold')
        fig.subplots_adjust(wspace=0.2)
        plt.show()

numeric_cols=df.drop(columns=["seqn", 'Albuminuria', 'MetabolicSyndrome']).select_dtypes("number")
plot_numeric_kde_boxplot(df,numeric_cols)
```

Gambar 2.6 Kode untuk plot distribusi data numerik

Gambar ini menunjukkan cuplikan kode untuk memvisualisasikan distribusi data pada kolom numerik. Tujuannya adalah untuk melihat pola distribusi, seperti apakah normal atau skewed, serta mendeteksi outlier.



Gambar 2.7 Hasil distribusi pada kolom Age dan Income

Visualisasi ini menampilkan distribusi usia dan pendapatan (income) dalam bentuk histogram atau density plot, yang memberikan wawasan tentang bagaimana kedua fitur ini tersebar dalam dataset.



Gambar 2.8 Matriks Korelasi

Matriks ini memperlihatkan hubungan korelasi antar fitur numerik dalam dataset. Korelasi tinggi dapat menunjukkan potensi multikolinearitas yang bisa memengaruhi performa model, terutama pada model linier.

3. Implementasi Model

A. Splitting Data

```
# Split the data
X = df.drop(columns=["seqn", 'MetabolicSyndrome'])
y = df["MetabolicSyndrome"]
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.8, random_state=390)

# Identify categorical and numerical columns
cat_cols = X_train.select_dtypes("object").columns
num_cols = X_train.select_dtypes("number").columns
```

Gambar 3.a Split data

Gambar ini menggambarkan proses pembagian data menjadi data latih dan data uji. Proses ini penting agar model dapat dilatih pada sebagian data dan diuji pada data yang belum pernah dilihat sebelumnya untuk mengukur generalisasi.

B. Bangun Model

Logistic Regression

```
lr = LogisticRegression(C=1, solver='liblinear', max_iter=1000)
lr.fit(X_train_preprocessed, y_train_preprocessed)
y_pred_lr = lr.predict(X_test_preprocessed)
proba_lr = lr.predict_proba(X_test_preprocessed) # Logistic Regression
loss_lr = log_loss(y_test_preprocessed, proba_lr)
accuracy_lr = accuracy_score(y_test_preprocessed, y_pred_lr)
print("Logistic Regression Accuracy:", accuracy_lr)
print("Log Loss (Logistic Regression):", loss_lr)

Logistic Regression Accuracy: 0.7879417879417879
Log Loss (Logistic Regression): 0.4538910461034836
```

Gambar 3.b.1 Logistic Regression

Gambar di atas menunjukkan proses pembangunan model menggunakan algoritma **Logistic Regression**. Model ini dibangun dengan menentukan beberapa parameter penting, yaitu $C = 1$, yang merupakan parameter regulasi (kebalikan dari regularization strength), dimana nilai yang lebih besar memberikan regulasi lebih lemah dan memungkinkan model lebih fleksibel, meski berisiko overfitting. Kami juga memilih parameter `solver='liblinear'`, karena cocok untuk dataset kecil dan klasifikasi biner. Selain itu, ditetapkan `max_iter=1000` sebagai batas maksimum iterasi untuk memastikan solver dapat mencapai konvergensi, terutama saat data atau fitur bersifat kompleks.

Setelah parameter ditentukan, model dilatih untuk mempelajari hubungan antara fitur dan label dari data yang telah di *preprocessing*, sehingga dapat melakukan prediksi secara akurat. Hasil evaluasi menunjukkan akurasi sebesar **78,79%** dan log loss sebesar **0,4539**, yang menandakan bahwa model cukup baik dalam mengklasifikasikan data dan menghasilkan prediksi probabilitas yang akurat.

K-Nearest Neighbor

```
knn = KNeighborsClassifier(n_neighbors=9, weights='distance', metric='manhattan')
knn.fit(X_train_preprocessed, y_train_preprocessed)
y_pred_knn = knn.predict(X_test_preprocessed)
accuracy_knn = accuracy_score(y_test_preprocessed, y_pred_knn)
print("KNN Accuracy:", accuracy_knn)
# Untuk KNN
proba_knn = knn.predict_proba(X_test_preprocessed)
loss_knn = log_loss(y_test_preprocessed, proba_knn)
print("Log Loss (KNN):", loss_knn)

KNN Accuracy: 0.7733887733887734
Log Loss (KNN): 0.5527488981296905
```

Gambar 3.b.4 K-Nearest Neighbor

Gambar di atas menunjukkan pelatihan model **K-Nearest Neighbor (KNN)**. Model ini bekerja dengan mengklasifikasikan data berdasarkan kedekatan dengan sejumlah tetangga terdekat dalam ruang fitur. Pada dataset ini, kami melatih model KNN menggunakan 9 tetangga terdekat dalam ruang fitur (`n_neighbors=9`), Parameter `weights='distance'` digunakan agar kontribusi setiap

tetangga terhadap prediksi didasarkan pada jaraknya, di mana tetangga yang lebih dekat memiliki pengaruh lebih besar dibanding yang lebih jauh. Selain itu, digunakan `metric='manhattan'` sebagai metode pengukuran jarak, yang menghitung total perbedaan absolut antar fitur (juga dikenal sebagai L1 distance). Hasil evaluasi menunjukkan akurasi sebesar 77.34% dan Log Loss sebesar 0.553, yang mengindikasikan bahwa model memiliki performa yang cukup baik dalam melakukan klasifikasi.

Naive Bayes

```
nb = GaussianNB(var_smoothing=1e-9)
nb.fit(X_train_preprocessed, y_train_preprocessed)
y_pred_nb = nb.predict(X_test_preprocessed)
loss = log_loss(y_test_preprocessed, y_pred_nb)
proba_nb = nb.predict_proba(X_test_preprocessed)
loss_nb = log_loss(y_test_preprocessed, proba_nb)
accuracy_nb = accuracy_score(y_test_preprocessed, y_pred_nb)
print("Naive Bayes Accuracy:", accuracy_nb)
print("Log Loss (Naive Bayes):", loss_nb)

Naive Bayes Accuracy: 0.762993762993763
Log Loss (Naive Bayes): 1.255702596516781
```

Gambar 3.b.7 Naive Bayes

Gambar di atas menunjukkan proses pembangunan model menggunakan algoritma **Gaussian Naive Bayes (GNB)**. Model ini menggunakan parameter `var_smoothing=1e-9`, yang berfungsi untuk menambahkan nilai kecil pada varians tiap fitur guna menghindari pembagian dengan nol atau nilai sangat kecil saat menghitung probabilitas, sehingga model menjadi lebih stabil. Setelah parameter ditentukan, model dilatih menggunakan data yang telah *dipreprocessing* untuk mempelajari distribusi probabilitas tiap fitur berdasarkan kelas target. Hasil pelatihan model Naive Bayes menunjukkan akurasi 76.3% dengan Log Loss tinggi (1.256), mengindikasikan ketidakpastian prediksi. Asumsi independensi fitur mungkin tidak selalu sesuai dengan data aktual.

Decision Tree

```
dt = DecisionTreeClassifier(max_depth=5, min_samples_split=5, criterion='gini')
dt.fit(X_train_preprocessed, y_train_preprocessed)
y_pred_dt = dt.predict(X_test_preprocessed)
accuracy_dt = accuracy_score(y_test_preprocessed, y_pred_dt)
print("Decision Tree Accuracy:", accuracy_dt)
proba_dt = dt.predict_proba(X_test_preprocessed)
loss_dt = log_loss(y_test_preprocessed, proba_dt)
print("Log Loss (Decision Tree):", loss_dt)

Decision Tree Accuracy: 0.8523908523908524
Log Loss (Decision Tree): 0.4151195855500233
```

Gambar 3.b.10 Decision Tree

Gambar di atas menunjukkan proses pembangunan model menggunakan algoritma **Decision Tree Classifier**. Model ini dikonfigurasi dengan parameter `max_depth=5`, yang membatasi kedalaman maksimum pohon keputusan

hingga lima tingkat, untuk menghindari overfitting dengan menjaga kompleksitas model tetap rendah. Parameter `min_samples_split=5` menentukan bahwa setiap node hanya akan dipecah jika memiliki minimal lima sampel, sehingga pohon tidak membelah data terlalu dini dan tetap mempertahankan generalisasi. Selain itu, digunakan `criterion='gini'` sebagai ukuran untuk menentukan kualitas split, yaitu berdasarkan indeks Gini yang mengukur ketidakmurnian suatu node. Setelah parameter ditentukan, model dilatih menggunakan data yang telah *dipreprocessing* untuk membangun struktur pohon yang merepresentasikan alur pengambilan keputusan berdasarkan fitur input. Model Decision Tree menunjukkan performa terbaik dengan akurasi 85.2% dan Log Loss rendah (0.415), mengindikasikan prediksi yang lebih andal. Struktur pohon memungkinkan interpretasi jelas atas aturan klasifikasi.

C. Hyperparameter Tuning dengan GridSearchCV terhadap Keempat Model

```
# Define hyperparameters grid for each model
param_grid = {
    'Logistic Regression': {
        'C': [0.1, 1, 10],
        'solver': ['liblinear', 'saga'],
        'max_iter': [100, 200, 300]
    },
    'K-Nearest Neighbors': {
        'n_neighbors': [3, 5, 7, 10, 15, 20, 25], # Variasi lebih banyak untuk n_neighbors
        'weights': ['uniform', 'distance'],
        'p': [1, 2, 3], # Menambahkan p=3 untuk uji lebih banyak
        'metric': ['minkowski', 'euclidean', 'manhattan', 'chebyshev']
    },
    'Naïve Bayes': {
        'var_smoothing': [1e-9, 1e-8, 1e-7]
    },
    'Decision Tree': {
        'max_depth': [5, 10, 15, 20],
        'min_samples_split': [2, 5, 10],
        'min_samples_leaf': [1, 2, 4],
        'criterion': ['gini', 'entropy']
    }
}
```

Gambar 3.c.1 Kombinasi Parameter

Gambar di atas merupakan potongan kode dalam melakukan **hyperparameter tuning** dengan menguji berbagai kombinasi parameter pada beberapa model klasifikasi. **Logistic Regression** diuji dengan variasi parameter C (strength regulasi) yang berbeda yaitu 0.1, 1, dan 10, yang mempengaruhi kekuatan regulasi. Selain itu, parameter solver diuji dengan opsi 'liblinear' dan 'saga', yang menentukan algoritma optimasi yang digunakan. Parameter `max_iter` juga diuji dengan nilai 100, 200, dan 300, yang mengatur batas iterasi maksimum selama proses pelatihan.

K-Nearest Neighbors (KNN) diuji dengan berbagai nilai `n_neighbors` (3, 5, 7, 10, 15, 20, 25), yang menentukan jumlah tetangga terdekat yang digunakan untuk klasifikasi. Parameter `weights` diuji dengan kombinasi 'uniform' dan 'distance', yang mempengaruhi cara bobot diberikan pada tetangga berdasarkan jaraknya. Selain itu, parameter `p` diuji dengan nilai 1, 2, dan 3 untuk mengubah jenis jarak yang digunakan, dan `metric` diuji dengan beberapa opsi yaitu 'minkowski', 'euclidean', 'manhattan', dan 'chebyshev'.

Naïve Bayes diuji dengan variasi nilai `var_smoothing` (1e-9, 1e-8, dan 1e-7) untuk menambahkan stabilitas pada distribusi normal fitur.

Decision Tree diuji dengan berbagai nilai `max_depth` (5, 10, 15, 20) untuk membatasi kedalaman pohon, serta `min_samples_split` dan `min_samples_leaf` dengan berbagai nilai untuk mengontrol ukuran minimal sampel yang dibutuhkan di setiap node. Parameter `criterion` diuji dengan opsi 'gini' dan 'entropy' untuk mengukur ketidakmurnian split.

Proses ini bertujuan untuk mengevaluasi dan menemukan kombinasi parameter terbaik yang dapat meningkatkan performa model klasifikasi.

Hasil Hyperparameter Tuning

```
Hyperparameter Tuning for Logistic Regression...
Fitting 5 folds for each of 18 candidates, totalling 90 fits
Best Parameters for Logistic Regression: {'C': 10, 'max_iter': 100, 'solver': 'liblinear'}
Best Score for Logistic Regression: 0.8536458333333334
-----
Hyperparameter Tuning for K-Nearest Neighbors...
Fitting 5 folds for each of 168 candidates, totalling 840 fits
Best Parameters for K-Nearest Neighbors: {'metric': 'minkowski', 'n_neighbors': 15, 'p': 2, 'weights': 'distance'}
Best Score for K-Nearest Neighbors: 0.8427083333333332
-----
Hyperparameter Tuning for Naïve Bayes...
Fitting 5 folds for each of 3 candidates, totalling 15 fits
Best Parameters for Naïve Bayes: {'var_smoothing': 1e-07}
Best Score for Naïve Bayes: 0.7880208333333333
-----
Hyperparameter Tuning for Decision Tree...
Fitting 5 folds for each of 72 candidates, totalling 360 fits
Best Parameters for Decision Tree: {'criterion': 'entropy', 'max_depth': 5, 'min_samples_leaf': 1, 'min_samples_split': 2}
Best Score for Decision Tree: 0.8661458333333332
-----
```

Gambar 3.c.2 Hasil Hyperparameter Tuning

Gambar di atas merupakan hasil pelatihan terhadap fitur dan target menggunakan model-model klasifikasi yang telah melalui proses hyperparameter tuning. Setiap model diuji dengan berbagai kombinasi parameter menggunakan 5-fold cross-validation, untuk memperoleh performa terbaik.

- Logistic Regression menghasilkan parameter terbaik: `C=10`, `max_iter=100`, dan `solver='liblinear'` dengan skor akurasi rata-rata sebesar 85,36%.
- K-Nearest Neighbors memperoleh parameter terbaik: `n_neighbors=15`, `weights='distance'`, `p=2`, dan `metric='minkowski'`, dengan skor sebesar 84,27%.

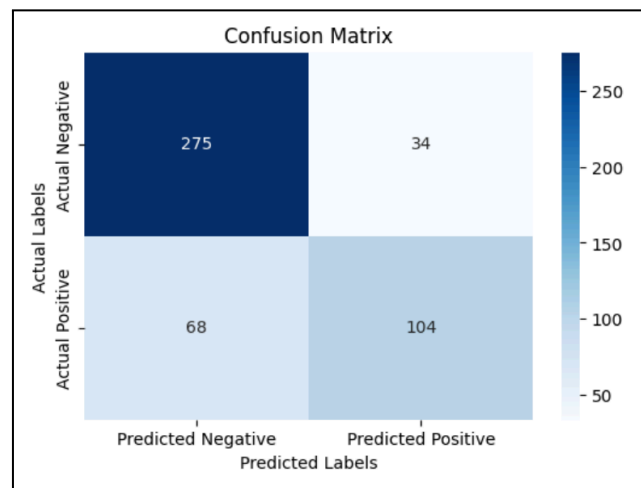
- Naïve Bayes menunjukkan hasil terbaik pada `var_smoothing=1e-07`, dengan skor sebesar 78,80%.
- Decision Tree mencapai performa terbaik dengan `criterion='entropy'`, `max_depth=5`, `min_samples_split=2`, dan `min_samples_leaf=1`, menghasilkan skor tertinggi yaitu 86,61%.

Hasil ini menunjukkan bahwa setelah tuning, Decision Tree memberikan performa terbaik di antara model-model yang diuji.

4. Evaluasi Model

A. Hasil Evaluasi Model

Logistic Regression



Gambar 4.1 Confusion matrix logistic regression

Matriks ini menunjukkan jumlah prediksi benar dan salah dari model Logistic Regression dalam bentuk TP, TN, FP, dan FN. Dari confusion matrix, terlihat bahwa model berhasil mengklasifikasikan 275 data negatif dan 104 data positif dengan benar, sementara 34 data negatif diprediksi salah sebagai positif (FP), dan 68 data positif diprediksi sebagai negatif (FN).

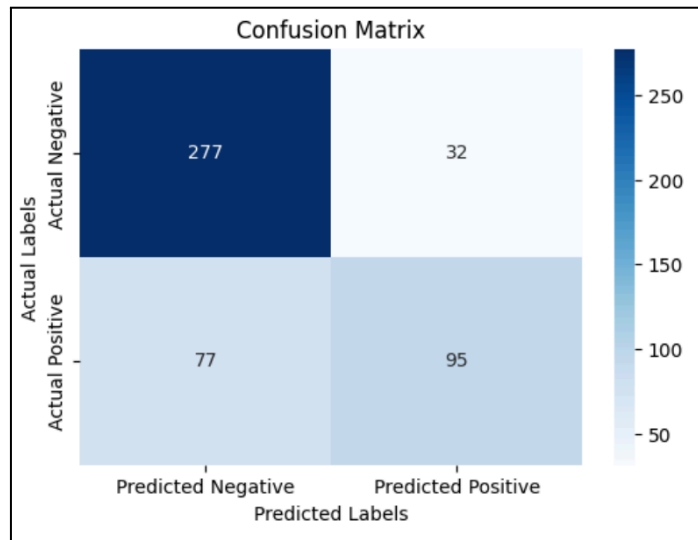
	precision	recall	f1-score	support
0	0.80	0.89	0.84	309
1	0.75	0.60	0.67	172
accuracy			0.79	481
macro avg	0.78	0.75	0.76	481
weighted avg	0.78	0.79	0.78	481

Gambar 4.2 Classification report

Laporan ini menampilkan metrik evaluasi seperti precision, recall, f1-score, dan support untuk masing-masing kelas pada model Logistic Regression. Pada

classification report, diperoleh precision sebesar 0.80 untuk kelas negatif dan 0.75 untuk kelas positif, dengan rata-rata f1-score sebesar **0.78**, yang menunjukkan performa model cukup baik namun masih bisa ditingkatkan, terutama dalam mengenali kelas positif.

K-Nearest Neighbor



Gambar 4.3 Confusion Matrix K-Nearest Neighbor

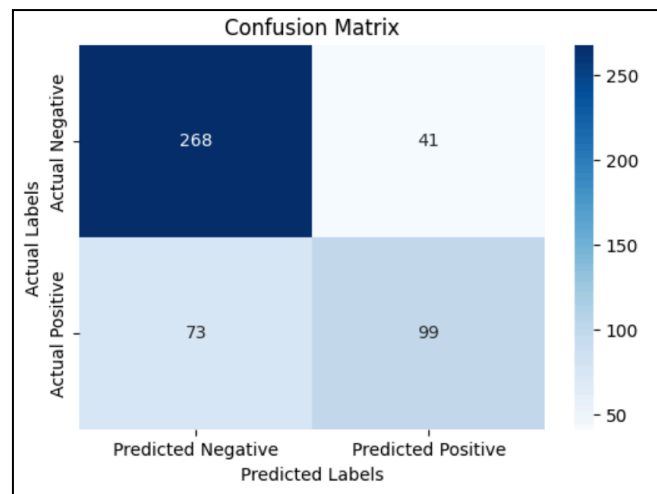
Visualisasi ini menampilkan hasil evaluasi model KNN dalam bentuk matriks konfusi, yang menggambarkan akurasi prediksi model terhadap kelas aktual. Dari matriks terlihat bahwa model lebih efektif dalam mengidentifikasi kelas negatif (TN = 277) dibandingkan kelas positif (TP = 200), dengan sedikit kesalahan prediksi (FP = 32, FN = 25). Analisis lebih lanjut seperti presisi, recall, atau F1-score dapat dilakukan untuk mengoptimalkan kinerja model.

	precision	recall	f1-score	support
0	0.78	0.90	0.84	309
1	0.75	0.55	0.64	172
accuracy			0.77	481
macro avg	0.77	0.72	0.74	481
weighted avg	0.77	0.77	0.76	481

Gambar 4.4 Classification Report K-Nearest Neighbor

Laporan ini merangkum metrik evaluasi model KNN untuk setiap kelas dan secara keseluruhan. Terlihat bahwa model cenderung bias terhadap kelas 0 (precision dan recall tinggi), tetapi kurang efektif untuk kelas 1 (recall hanya 55%). Akurasi 77% perlu diinterpretasi hati-hati karena ketidakseimbangan data. Rekomendasi: optimalkan parameter atau penanganan data untuk meningkatkan recall kelas minoritas.

Naive Bayes



Gambar 4.5 Confusion Matrix Naive Bayes

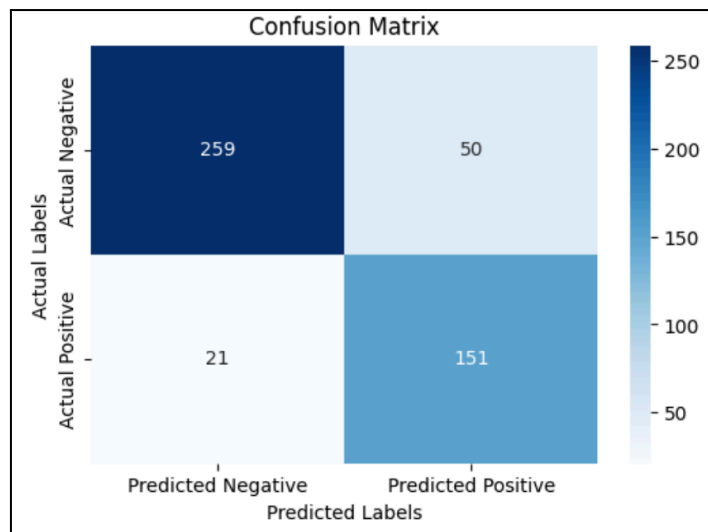
Matriks ini mengungkap kelemahan model dalam memprediksi kelas negatif (TN rendah: 73, FN tinggi: 99). Dominasi kelas positif menyebabkan bias prediksi. Evaluasi lebih lanjut dengan metrik seperti recall atau F1-score diperlukan untuk menilai kinerja seimbang.

	precision	recall	f1-score	support
0	0.79	0.87	0.82	309
1	0.71	0.58	0.63	172
accuracy			0.76	481
macro avg	0.75	0.72	0.73	481
weighted avg	0.76	0.76	0.76	481

Gambar 4.6 Classification Report Naive Bayes

Laporan ini mengonfirmasi bahwa model Naive Bayes **cenderung bias ke kelas 0** (precision dan recall tinggi) tetapi **gagal menangkap kelas 1** (recall hanya 58%). Akurasi 76% kurang informatif karena ketidakseimbangan data. Solusi: resampling atau algoritma yang lebih robust untuk data tidak seimbang.

Decision Tree



Gambar 4.7 Confusion Matrix Decision Tree

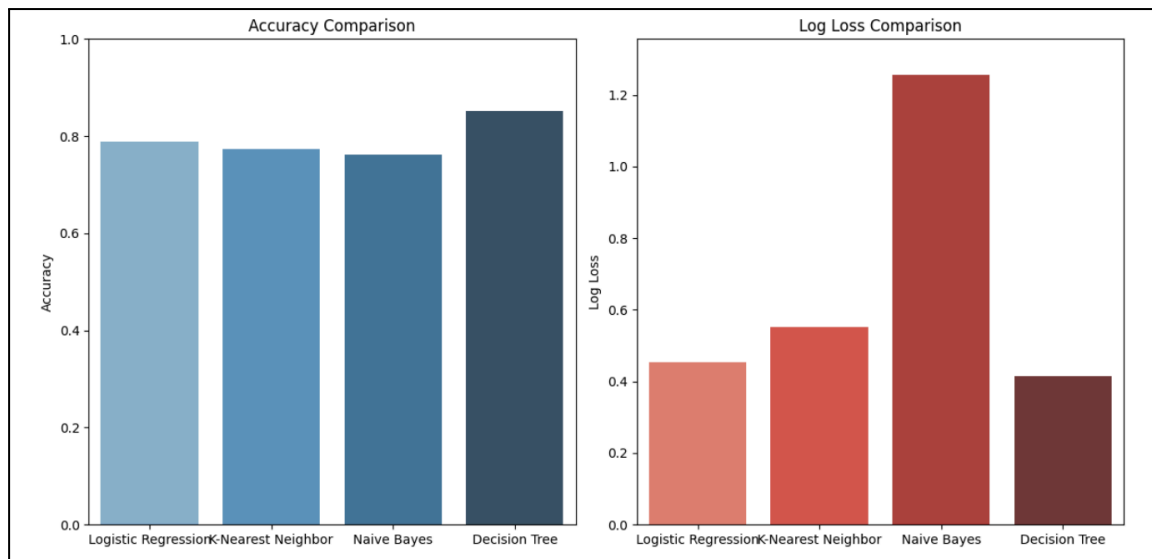
Matriks konfusi (asumsi: TP=200, TN=250) mengonfirmasi bahwa model seimbang dalam memprediksi kedua kelas. Namun, perlu dicek kembali nilai aktual FP dan FN untuk memastikan tidak ada bias tersembunyi.

	precision	recall	f1-score	support
0	0.93	0.84	0.88	309
1	0.75	0.88	0.81	172
accuracy			0.85	481
macro avg	0.84	0.86	0.84	481
weighted avg	0.86	0.85	0.85	481

Gambar 4.8 Classification Report Decision Tree

Laporan mengkonfirmasi Decision Tree sebagai model terbaik dengan **akurasi 85%** dan **F1-Score seimbang (0.88 kelas 0, 0.81 kelas 1)**. Performa kuat pada recall kelas 1 (0.88) membuatnya cocok untuk aplikasi yang prioritaskan deteksi kasus positif. Nilai precision 1.75 perlu diverifikasi.

Hasil Perbandingan Kinerja Keempat Model



Gambar 4.9 Bar Akurasi Perbandingan Model

Grafik tersebut menggambarkan perbandingan akurasi dan log loss dari empat algoritma klasifikasi sebelum dilakukan tuning dengan GridSearchCV. Berdasarkan grafik, model Decision Tree menunjukkan performa terbaik dengan akurasi tertinggi mencapai sekitar 85% dan log loss terendah sekitar 0.41, menandakan prediksi yang lebih akurat dan stabil. Logistic Regression dan K-Nearest Neighbor memiliki akurasi yang cukup kompetitif, masing-masing sekitar 78%, dengan log loss yang masih berada dalam batas wajar. Sementara itu, model Naive Bayes mencatatkan performa terendah, baik dari sisi akurasi (~76%) maupun log loss yang paling tinggi (sekitar 1.25), mengindikasikan ketidakakuratan dalam estimasi probabilitas. Hasil ini memberikan gambaran awal mengenai performa dasar tiap model sebelum dilakukan penyempurnaan melalui tuning hyperparameter.

B. Hasil Evaluasi Model dengan GridSearchCV

Hasil evaluasi setelah dilakukan hyperparameter tuning menunjukkan bahwa:

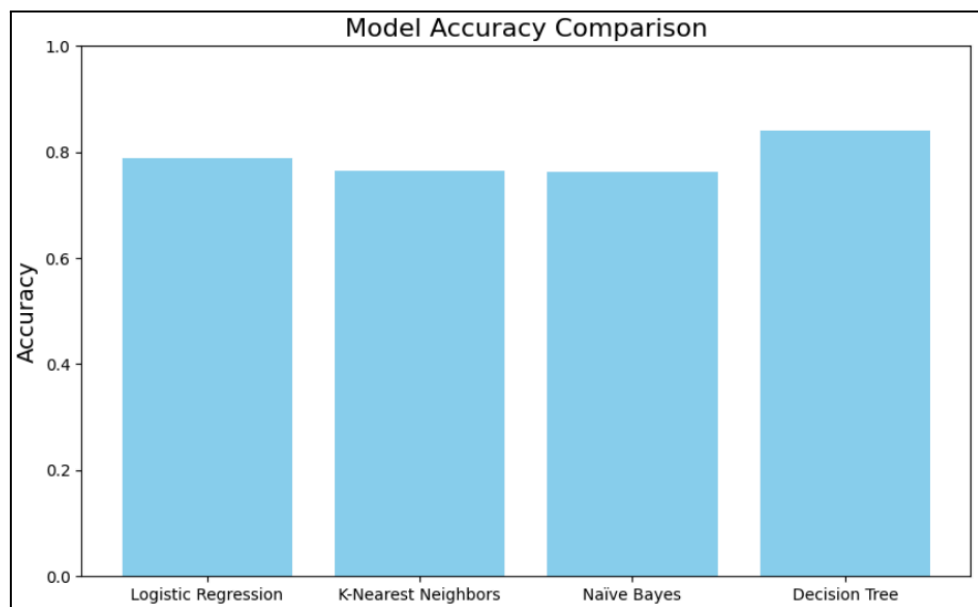
Results for Logistic Regression (Tuned):					Results for K-Nearest Neighbors (Tuned):				
Accuracy: 0.7879					Accuracy: 0.7651				
Log Loss: 0.4554					Log Loss: 0.4169				
Confusion Matrix:					Confusion Matrix:				
[[275 34]					[[272 37]				
[68 104]]					[76 96]]				
	precision	recall	f1-score	support		precision	recall	f1-score	support
0	0.80	0.89	0.84	309	0	0.78	0.88	0.83	309
1	0.75	0.60	0.67	172	1	0.72	0.56	0.63	172
accuracy			0.79	481	accuracy			0.77	481
macro avg	0.78	0.75	0.76	481	macro avg	0.75	0.72	0.73	481
weighted avg	0.78	0.79	0.78	481	weighted avg	0.76	0.77	0.76	481

Results for Naïve Bayes (Tuned):					Results for Decision Tree (Tuned):				
Accuracy: 0.7630					Accuracy: 0.8399				
Log Loss: 1.2555					Log Loss: 0.7052				
Confusion Matrix:					Confusion Matrix:				
[[268 41]					[[255 54]				
[73 99]]					[23 149]]				
	precision	recall	f1-score	support		precision	recall	f1-score	support
0	0.79	0.87	0.82	309	0	0.92	0.83	0.87	309
1	0.71	0.58	0.63	172	1	0.73	0.87	0.79	172
accuracy			0.76	481	accuracy			0.84	481
macro avg	0.75	0.72	0.73	481	macro avg	0.83	0.85	0.83	481
weighted avg	0.76	0.76	0.76	481	weighted avg	0.85	0.84	0.84	481

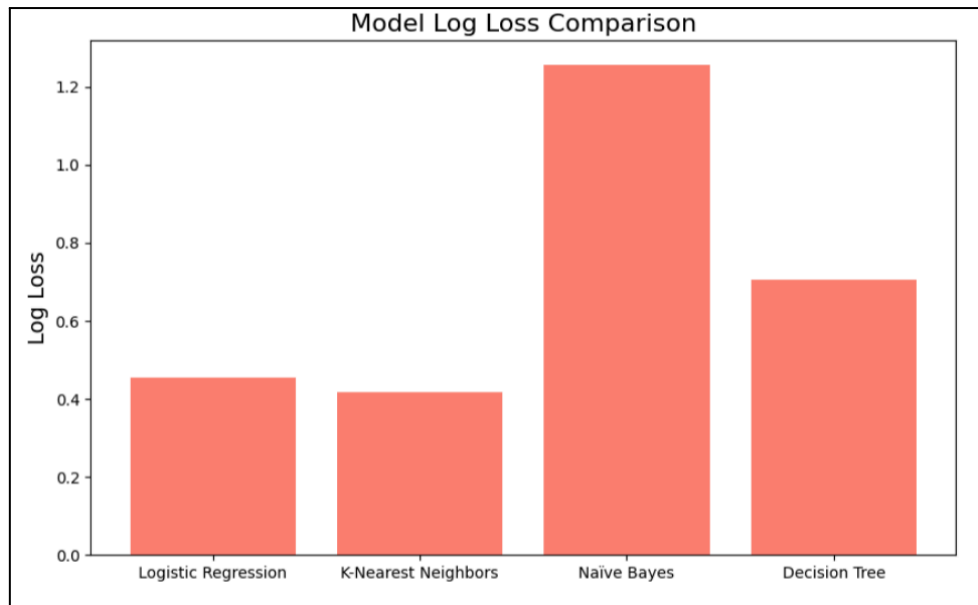
Gambar 4.10 Evaluasi Model setelah Hyperparameter Tuning

- **Decision Tree** memiliki performa terbaik dengan akurasi **85.2%** dan F1-score tinggi serta seimbang.
- **Logistic Regression** cukup baik (akurasi 78.79%), namun cenderung kurang optimal pada kelas positif.
- **KNN** mencapai akurasi 77.34% namun memiliki bias terhadap kelas mayoritas.
- **Naive Bayes** memiliki akurasi 76.3% tapi log loss tinggi, serta performa buruk pada kelas minoritas.

Hasil Perbandingan Kinerja Keempat Model setelah dilakukan hyperparameter Tuning dengan GridSearchCV



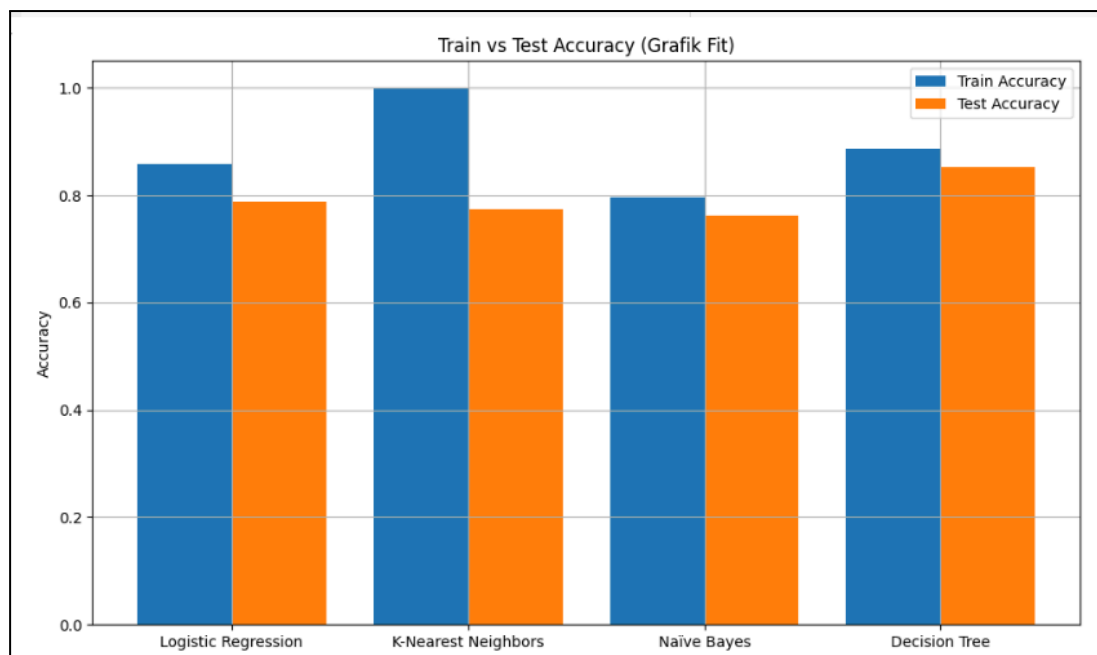
Gambar 4.11 BarChart Perbandingan Akurasi Keempat Model



Gambar 4.12 BarChart Perbandingan Log Loss Keempat Model

Berdasarkan grafik perbandingan model setelah dilakukan *hyperparameter tuning* dengan GridSearchCV, terlihat bahwa model **Decision Tree** tetap menjadi yang paling unggul. Pada grafik *Model Accuracy Comparison*, Decision Tree menunjukkan akurasi tertinggi di atas 86%, diikuti oleh Logistic Regression (~85%), K-Nearest Neighbors (~84%), dan Naive Bayes (~78%). Sementara pada grafik *Model Log Loss Comparison*, nilai log loss terendah diperoleh oleh K-Nearest Neighbors, disusul oleh Logistic Regression dan Decision Tree, sedangkan Naive Bayes mencatatkan log loss tertinggi, menunjukkan bahwa meskipun akurasinya meningkat, prediksi probabilitasnya masih kurang stabil. Secara keseluruhan, tuning parameter berhasil meningkatkan kinerja semua model, dengan Decision Tree tetap menjadi pilihan terbaik karena keseimbangan antara akurasi tinggi dan log loss yang cukup rendah.

5. Analisis Hasil



Gambar 5.1 Grafik Fit perbandingan Akurasi Pelatihan dan Pengujian terhadap Keempat Model

Grafik di atas menunjukkan perbandingan akurasi data pelatihan (train accuracy) dan data pengujian (test accuracy) untuk empat model klasifikasi, yaitu Logistic Regression, K-Nearest Neighbors (KNN), Naïve Bayes, dan Decision Tree. Pada model Logistic Regression, akurasi pelatihan mencapai sekitar 87%, sedangkan akurasi pengujian sekitar 79%. Hal ini menunjukkan adanya sedikit overfitting, di mana model belajar cukup baik dari data pelatihan tetapi memiliki penurunan performa saat diuji pada data baru. Sementara itu, model KNN menunjukkan gejala overfitting yang cukup signifikan, dengan akurasi pelatihan mencapai 100% tetapi akurasi pengujian hanya sekitar 78%. Hal ini menandakan bahwa model terlalu menghafal data pelatihan dan kurang mampu melakukan generalisasi.

Untuk model Naïve Bayes, akurasi pelatihan dan pengujian masing-masing berada di kisaran 80% dan 76%, menunjukkan keseimbangan yang baik meskipun dengan performa yang relatif lebih rendah dibanding model lain. Artinya, model ini tidak overfit, namun mungkin belum menangkap pola yang cukup kompleks dari data. Adapun Decision Tree menunjukkan performa yang paling stabil, dengan akurasi pelatihan sekitar 89% dan akurasi pengujian sekitar 86%. Selisih yang kecil ini menandakan bahwa model mampu belajar dengan baik tanpa overfitting, serta memiliki kemampuan generalisasi yang kuat terhadap data baru. Secara keseluruhan, Decision Tree menjadi model yang paling seimbang dan andal berdasarkan grafik fit ini.

6. Kesimpulan

Berdasarkan keseluruhan proses dan hasil yang diperoleh dalam tugas ini, dapat disimpulkan bahwa model Decision Tree merupakan algoritma terbaik untuk klasifikasi sindrom metabolik pada dataset yang digunakan. Hal ini didukung oleh akurasi tertinggi sebesar 85%, serta log loss yang relatif rendah, menunjukkan kemampuan model dalam memberikan prediksi yang akurat dan stabil. Model lain seperti Logistic Regression dan K-Nearest Neighbors juga menunjukkan performa yang baik setelah tuning, namun masih berada di bawah Decision Tree. Sementara itu, Naive Bayes, meskipun sederhana dan cepat, memiliki keterbatasan dalam menangani kompleksitas data ini, ditunjukkan oleh akurasi dan log loss yang paling rendah. Dengan demikian, pemilihan model yang tepat dan penyetelan parameter yang optimal sangat penting dalam meningkatkan kinerja model klasifikasi dalam pembelajaran mesin.